

Diagrama de Clases POO

Jesus Santiago Lozano Peña

Ingeniería de Software

William Alexander Matallana

Universidad de Cundinamarca

Introducción

En el desarrollo de software, la correcta planificación de la estructura del sistema es fundamental para garantizar mantenibilidad, escalabilidad y claridad en el código. La Programación Orientada a Objetos (POO) propone modelar los programas a partir de clases y objetos que representan entidades del mundo real. Para facilitar este modelado se emplea el Lenguaje Unificado de Modelado (UML), dentro del cual el diagrama de clases es una de las herramientas más utilizadas. Este trabajo describe la teoría básica del diagrama de clases, sus principales relaciones y un ejemplo práctico.

¿Qué es un diagrama de clases?

Un diagrama de clases es una representación estática del sistema que muestra la estructura interna del software. En él se describen las clases, sus atributos, métodos y las relaciones entre ellas. Cada clase se representa mediante un rectángulo dividido en nombre, atributos y métodos, permitiendo visualizar el diseño antes de programar.

Relaciones entre clases

Las relaciones en un diagrama de clases permiten representar la manera en que los distintos elementos del sistema interactúan entre sí. Estas conexiones ayudan a comprender la estructura del software, la distribución de

responsabilidades y el flujo de información entre objetos. A continuación, se describen las relaciones más utilizadas en la Programación Orientada a Objetos.

Asociación

La asociación es la relación más básica entre clases. Indica que un objeto puede comunicarse o interactuar con otro para cumplir una determinada función. No implica dependencia fuerte ni propiedad, simplemente existe un vínculo de uso o conocimiento.

Por ejemplo, en un sistema académico, un Estudiante se asocia con un Curso, ya que el estudiante puede inscribirse en uno o varios cursos. Ambas clases pueden existir de manera independiente.

Esta relación suele representarse con una línea simple entre las clases y puede incluir multiplicidad (1, 1..*, *), lo que indica cuántos objetos participan en la relación.

Agregación

La agregación es un tipo especial de asociación que representa una relación “todo–parte” débil. En este caso, una clase contiene a otras, pero las partes pueden existir de forma independiente del todo.

Por ejemplo, un Equipo está formado por Jugadores. Si el equipo se disuelve, los jugadores continúan existiendo y pueden formar parte de otro equipo.

Se representa gráficamente con un rombo blanco en el extremo de la clase que actúa como contenedor. Esta relación es útil cuando se desea modelar agrupaciones sin dependencia estricta entre los objetos.

Composición

La composición también representa una relación “todo–parte”, pero con una dependencia fuerte entre los objetos. Las partes no pueden existir sin el objeto principal que las contiene. Si el todo desaparece, sus componentes también.

Un ejemplo común es una Casa compuesta por Habitaciones. Si la casa deja de existir, las habitaciones no tienen sentido por sí solas dentro del modelo.

En UML se representa mediante un rombo negro. Esta relación se utiliza cuando los elementos forman una estructura inseparable dentro del sistema.

Herencia (Generalización)

La herencia, también conocida como generalización, permite que una clase hija adquiera los atributos y métodos de una clase padre. Esto favorece la reutilización de código y la creación de jerarquías lógicas basadas en características comunes.

Por ejemplo, una clase Animal puede tener atributos como edad y peso, y métodos como comer(). Las clases Perro y Gato pueden heredar estas propiedades y añadir comportamientos específicos.

La herencia representa una relación del tipo “es un”, ya que un perro es un animal. En UML se muestra con una flecha que apunta desde la clase hija hacia la clase padre.

Dependencia

La dependencia ocurre cuando una clase utiliza a otra de forma temporal, por ejemplo, como parámetro en un método, variable local o llamada a una función específica. No existe una relación permanente entre los objetos, pero uno necesita del otro para realizar una tarea concreta.

Por ejemplo, una clase Factura puede depender de CalculadoraImpuestos únicamente para calcular un valor durante la ejecución de un método.

Se representa con una línea discontinua y una flecha. Este tipo de relación indica bajo acoplamiento y suele ser deseable para mantener sistemas más flexibles y fáciles de modificar.

Ejemplo práctico en POO

Ejemplo: Sistema de gestión de una universidad

Para ilustrar el uso de los diagramas de clases y las relaciones en la Programación Orientada a Objetos, se propone el diseño de un sistema básico de gestión universitaria.

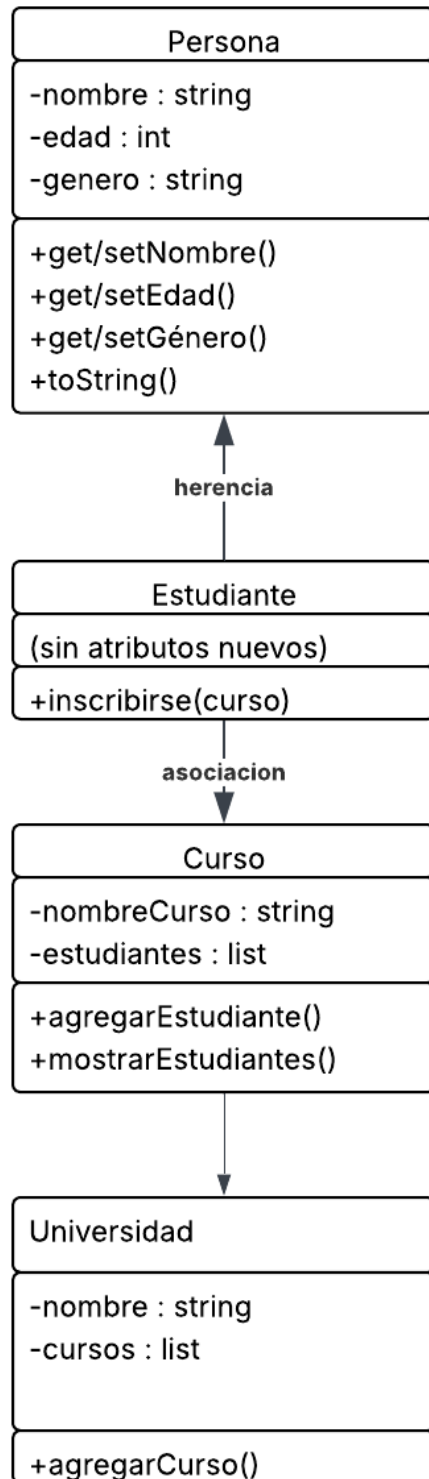
El sistema debe permitir administrar estudiantes, cursos y la universidad como entidad principal.

Se identifican las siguientes clases:

- Universidad
- Estudiante
- Curso
- Persona (clase base)

Relaciones presentes

- Herencia: Estudiante hereda de Persona.
- Composición: Universidad contiene cursos (si la universidad desaparece, los cursos también).
- Agregación: Curso agrupa estudiantes, pero estos pueden existir fuera del curso.
- Asociación: Estudiante se inscribe en cursos.
- Dependencia: Curso usa un método externo para calcular promedio.



Conclusión

Los diagramas de clases permiten planificar y estructurar sistemas orientados a objetos de manera clara y organizada. Su uso mejora la calidad del diseño, facilita la comunicación entre desarrolladores y reduce errores antes de la implementación.

Fuentes de consulta

Booch, G., Rumbaugh, J., & Jacobson, I. El Lenguaje Unificado de Modelado
Sommerville, I. Ingeniería de Software. Pearson.